

Factory Pattern — Class 2

Lesson 6: Reflection Factory

Objective

We understand how objects can be created dynamically by class name.

Why It Exists

Sometimes we do not want to hard-code this:

```
registry.put("PDF", PdfExportFile::new);  
registry.put("EXCEL", ExcelExportFile::new);
```

Reflection allows Java to create an object from class metadata.

Example

```
public class ReflectionExportFactory {  
  
    public ExportFile create(String className) {  
        try {  
            Class<?> clazz = Class.forName(className);  
            return (ExportFile) clazz.getDeclaredConstructor().newInstance();  
        } catch (Exception e) {  
            throw new RuntimeException("Cannot create export file", e);  
        }  
    }  
}
```

Usage:

```
ExportFile exportFile = factory.create(
```

```
"com.school.export.PdfExportFile"  
);  
exportFile.export();
```

Problem It Solves

It removes direct dependency from factory to concrete classes.

Common Mistake

Do not overuse Reflection Factory.

Problems:

Runtime error instead of compile-time error.

Harder to debug.

Harder to refactor.

Not beginner-friendly.

Reflection is powerful, but should be used carefully.

Lesson 7: Factory Method Pattern

Objective

We understand the official GoF Factory Method Pattern.

Why It Exists

Simple Factory has one factory class that decides everything.

Factory Method lets subclasses decide which object to create.

Product Interface

```
public interface Document {  
    void open();  
}
```

```
public class PdfDocument implements Document {  
    @Override  
    public void open() {  
        System.out.println("Open PDF document");  
    }  
}
```

```
public class WordDocument implements Document {  
    @Override  
    public void open() {  
        System.out.println("Open Word document");  
    }  
}
```

Creator Abstract Class

```
public abstract class DocumentApplication {  
  
    public void processDocument() {  
        Document document = createDocument();  
        document.open();  
    }  
  
    protected abstract Document createDocument();  
}
```

Concrete Creators

```
public class PdfApplication extends DocumentApplication {  
  
    @Override  
    protected Document createDocument() {  
        return new PdfDocument();  
    }  
}
```

```
public class WordApplication extends DocumentApplication {  
  
    @Override  
    protected Document createDocument() {  
        return new WordDocument();  
    }  
}
```

Usage

```
DocumentApplication app = new PdfApplication();  
app.processDocument();
```

Problem It Solves

The parent class controls the workflow.

The child class controls object creation.

This follows OCP because we can add a new document type by creating a new subclass, instead of modifying existing workflow code.

Common Mistake

Many developers confuse Simple Factory with Factory Method.

Simple Factory:

```
factory.create("PDF");
```

Factory Method:

```
new PdfApplication().processDocument();
```

Factory Method uses inheritance.

Simple Factory usually uses condition or map.

Lesson 8: Abstract Factory

Objective

We understand how to create families of related objects.

Why It Exists

Factory Method creates one product.

Abstract Factory creates a family of related products.

Example:

For hotel system:

Mobile UI needs:

MobileButton

MobileInput

MobileDialog

Web UI needs:

WebButton
WebInput
WebDialog

We want to make sure we do not mix Mobile component with Web component accidentally.

Product Interfaces

```
public interface Button {  
    void render();  
}
```

```
public interface Input {  
    void render();  
}
```

Concrete Products

```
public class WebButton implements Button {  
    @Override  
    public void render() {  
        System.out.println("Render web button");  
    }  
}
```

```
public class WebInput implements Input {  
    @Override  
    public void render() {  
        System.out.println("Render web input");  
    }  
}
```

```
public class MobileButton implements Button {  
    @Override  
    public void render() {  
        System.out.println("Render mobile button");  
    }  
}
```

```
public class MobileInput implements Input {  
    @Override  
    public void render() {  
        System.out.println("Render mobile input");  
    }  
}
```

Abstract Factory Interface

```
public interface UIFactory {  
    Button createButton();  
    Input createInput();  
}
```

Concrete Factories

```
public class WebUIFactory implements UIFactory {  
  
    @Override  
    public Button createButton() {  
        return new WebButton();  
    }  
  
    @Override  
    public Input createInput() {
```

```
        return new WebInput();
    }
}
```

```
public class MobileUIFactory implements UIFactory {
```

```
    @Override
    public Button createButton() {
        return new MobileButton();
    }
```

```
    @Override
    public Input createInput() {
        return new MobileInput();
    }
}
```

Client Code

```
public class LoginPage {

    private final Button button;
    private final Input input;

    public LoginPage(UIFactory factory) {
        this.button = factory.createButton();
        this.input = factory.createInput();
    }

    public void render() {
        input.render();
        button.render();
    }
}
```


Usage:

```
UIFactory factory = new WebUIFactory();  
LoginPage page = new LoginPage(factory);  
page.render();
```

Problem It Solves

Abstract Factory keeps related objects consistent.

Example:

WebFactory creates only Web components.
MobileFactory creates only Mobile components.

Common Mistake

Do not use Abstract Factory for one object only.

Bad use:

```
PaymentFactory.createPayment();
```

That is not Abstract Factory.

Abstract Factory is useful when we create multiple related objects.

Lesson 9: Factory in Java

Objective

We recognize Factory Pattern inside Java itself.

Example 1: Calendar

```
Calendar calendar = Calendar.getInstance();
```

We do not write:

```
new GregorianCalendar();
```

Java decides the correct implementation.

Example 2: NumberFormat

```
NumberFormat formatter = NumberFormat.getCurrencyInstance();
```

Java creates the correct formatter based on locale.

Example 3: Executors

```
ExecutorService executor = Executors.newFixedThreadPool(5);
```

Executors is a factory class that creates different executor implementations.

Getting Point

Factory is not only theory.

Java uses Factory many times to hide complex object creation.

Lesson 10: Factory in Spring

Objective

We understand that Spring itself is heavily based on object creation and dependency injection.

Example 1: @Bean

```
@Configuration
public class AppConfig {

    @Bean
    public PaymentService paymentService() {
        return new PaymentService();
    }
}
```

This method is factory-like.

Spring calls this method and manages the object.

Example 2: Spring Injecting Map of Strategies

```
@Component("ABA")
public class ABAPaymentStrategy implements PaymentStrategy {
    @Override
    public void pay(BigDecimal amount) {
        System.out.println("Pay by ABA");
    }
}
```

```
@Component("KHQR")
public class KhqrPaymentStrategy implements PaymentStrategy {
    @Override
    public void pay(BigDecimal amount) {
        System.out.println("Pay by KHQR");
    }
}
```

```

@Service
@RequiredArgsConstructor
public class PaymentStrategyFactory {

    private final Map<String, PaymentStrategy> strategies;

    public PaymentStrategy getStrategy(String method) {
        PaymentStrategy strategy = strategies.get(method);

        if (strategy == null) {
            throw new IllegalArgumentException("Unsupported payment
method");
        }

        return strategy;
    }
}

```

Service

```

@Service
@RequiredArgsConstructor
public class PaymentService {

    private final PaymentStrategyFactory factory;

    public void pay(String method, BigDecimal amount) {
        PaymentStrategy strategy = factory.getStrategy(method);
        strategy.pay(amount);
    }
}

```

Why This Is Good

Adding new payment method:

```
@Component("WING")
public class WingPaymentStrategy implements PaymentStrategy {
    @Override
    public void pay(BigDecimal amount) {
        System.out.println("Pay by Wing");
    }
}
```

No need to modify PaymentService.

No need to modify PaymentStrategyFactory.

This is OCP in real Spring Boot.

Lesson 11: Homework Project

Project: Export File System

Requirement

Build an export system that supports:

PDF
EXCEL
CSV
JSON

Each export type should have different behavior.

Step 1: Create Interface

```
public interface ExportFile {
    void export(String content);
}
```

Step 2: Create Implementations

```
@Component("PDF")
public class PdfExportFile implements ExportFile {
    @Override
    public void export(String content) {
        System.out.println("Export PDF: " + content);
    }
}
```

```
@Component("EXCEL")
public class ExcelExportFile implements ExportFile {
    @Override
    public void export(String content) {
        System.out.println("Export Excel: " + content);
    }
}
```

```
@Component("CSV")
public class CsvExportFile implements ExportFile {
    @Override
    public void export(String content) {
        System.out.println("Export CSV: " + content);
    }
}
```

Step 3: Create Factory

```
@Component
@RequiredArgsConstructor
public class ExportFileFactory {
```

```

private final Map<String, ExportFile> exportFiles;

public ExportFile getExportFile(String type) {
    ExportFile exportFile = exportFiles.get(type);

    if (exportFile == null) {
        throw new IllegalArgumentException("Unsupported export type: " +
type);
    }

    return exportFile;
}
}

```

Step 4: Create Service

```

@Service
@RequiredArgsConstructor
public class ExportService {

    private final ExportFileFactory factory;

    public void export(String type, String content) {
        ExportFile exportFile = factory.getExportFile(type);
        exportFile.export(content);
    }
}

```

Step 5: Add New Export Type

```

@Component("JSON")
public class JsonExportFile implements ExportFile {
    @Override

```

```
public void export(String content) {  
    System.out.println("Export JSON: " + content);  
}  
}
```

No need to modify:

ExportService
ExportFileFactory

Lesson 12: Common Mistakes

Mistake 1: Using Factory Everywhere

Not every new needs Factory.

This is fine:

```
User user = new User();
```

Factory is useful when object creation has decision logic or complexity.

Mistake 2: Putting Business Logic in Factory

Bad:

```
public class PaymentFactory {  
  
    public PaymentStrategy create(String method, BigDecimal amount) {  
        if (amount.compareTo(BigDecimal.valueOf(1000)) > 0) {  
            // business rule here  
        }  
  
        return new AbaPaymentStrategy();  
    }  
}
```



```
}  
}
```

Factory should create or select objects.

Business rules should stay in service, strategy, or policy classes.

Mistake 3: Confusing Strategy and Factory

Strategy:

Executes behavior.

Factory:

Creates or selects object.

Usually they work together.

Mistake 4: Overusing Reflection

Reflection looks advanced, but it can make code harder to debug.

Use it only when there is a strong reason.

Summary

Factory Pattern is a Creational Design Pattern.

It focuses on object creation.

It helps separate object creation from business logic.

Simple Factory is easiest to learn first.

Factory Method is the official GoF pattern.

Abstract Factory is for families of related objects.

Spring uses factory ideas everywhere through beans and dependency injection.

Factory is very useful after Strategy Pattern because Strategy gives us many implementations, and Factory helps us choose the correct one.